Recuperação do Trabalho II

Modo de entrega:

1. **Criar uma conta no github (https://github.com/)**
2. **Criar um repositório para o trabalho**
3. **Deixar o repositório como público, na entrega (para que eu possa visualizar)**
4. ***Commitar* com comentários explicativos a cada alteração importante (4 linhas de código já podem ser importantes)**
5. **Enviar um vídeo, de 5 minutos, explicando a lógica utilizada. Pode gravar apenas a tela do PC, usando algum software (OBS, por exemplo) ou com smartphone.**
6. **Trabalho Individual.**

Data de entrega: 17/10/2026
Nota máxima: 6, substituindo a nota obtida no trabalho II

Abaixo a descrição do trabalho, elaborada usando ajuda do Gemini.

Trabalho Prático: Sistema de Gestão e Simulação de Frota Logística em C

Objetivos do Trabalho

Aplicar na prática os conceitos fundamentais da linguagem C: entrada e saída formatada, estruturas de decisão, laços de repetição (for, while, do-while), manipulação de vetores (arrays unidimensionais), matrizes (arrays bidimensionais) e estruturas (struct), além da organização lógica do código.

1. Descrição do Problema

Uma empresa de logística necessita de um sistema em modo terminal para gerenciar seus veículos, motoristas e simular o consumo/tempo de entrega em uma malha rodoviária entre cidades da região.

O programa deve funcionar por meio de um menu interativo e manter todos os dados armazenados na memória durante a execução do programa (sem uso de arquivos ou alocação dinâmica avançada, apenas arrays fixos de structs).

2. Requisitos Estruturais e de Dados

A. Estruturas de Dados (struct)

Você deve definir obrigatoriamente as seguintes estruturas:

1. **struct Veiculo:**
    - **int id (Identificador único, ex: 101, 102)**

    - **char placa[8]**
    - **char modelo[30]**
    - **float consumoKmL (Quantos km o veículo faz por litro)**
    - **float capacidadeCarga (Em toneladas)**
    - **int status (0 = Disponível, 1 = Em Viagem, 2 = Em Manutenção)**
2. **struct Viagem:**
    - **int idVeiculo**
    - **int cidadeOrigem (Índice da cidade de 0 a $N - 1$)**
    - **int cidadeDestino (Índice da cidade de 0 a $N - 1$)**
    - **float distanciaTotal**
    - **float consumoEstimadoLitros**
    - **float custoEstimadoR$**

B. Matriz de Distâncias (Grafo de Cidades)

O sistema deve possuir uma matriz bidimensional estática $N \times N$ (onde $N = 5$ cidades) representando a distância em quilômetros entre cada par de cidades.

- **Se matriz[i][j] == 0, significa que as cidades são a mesma.**
- **Se matriz[i][j] == -1, significa que não há rota direta entre as cidades.**
- **Caso contrário, o valor representa a distância em km.**

3. Funcionalidades Obrigatórias (O Menu)

O programa deve executar em um laço contínuo até que a opção de saída seja escolhida.

==========

SISTEMA DE GESTÃO DE FROTA (C)

==========

1. Cadastrar Veículo

2. Listar Veículos

3. Consultar Matriz de Distâncias

4. Planejar / Simular Viagem

5. Relatório de Desempenho da Frota

6. Sair

==========

Detalhamento dos Módulos:

1. **Cadastro de Veículos:**
    - **Permite cadastrar até MAX_VEICULOS (ex: 10 veículos).**
    - **Deve validar se o ID não é duplicado e se a capacidade/consumo são valores positivos maiores que zero (usando laço do-while).**
2. **Listagem de Veículos:**
    - **Exibe em formato de tabela formatada todos os veículos cadastrados e seus respetivos status (convertendo 0, 1, 2 para texto compreensível como "Disponível").**
3. **Exibição e Atualização da Matriz de Distâncias:**
    - **Exibe a matriz 5×5 de cidades com cabeçalhos adequados.**
    - **Permite alterar a distância entre duas cidades específicas digitadas pelo usuário (garantindo simetria: se atualizar de A para B, atualiza de B para A).**
4. **Simulador de Viagem (Cálculos e Validações):**
    - **O usuário escolhe um veículo pelo ID e informa a cidade de Origem e Destino (de 0 a 4).**
    - **O sistema valida:**
        - **Se o veículo existe e se está 0 - Disponível.**
        - **Se há rota válida na matriz (distância > 0).**
    - **O sistema calcula:**
        - $Consumo(Litros) = \frac{Distância}{ConsumoKmL}$
        - **$\text{Custo Total (R\$)} = \text{Consumo (Litros)} \times \text{Preço do Combustível}$ (Assumir valor fixo de R$ 5,80/L via #define).**
    - **Se o usuário confirmar a viagem, o status do veículo muda para 1 - Em Viagem.**
5. **Relatórios e Estatísticas (Laços de Repetição Avançados):**
    - **Calcula e exibe:**
        - **O veículo com maior eficiência (maior km/l).**
        - **Quantidade e percentual de veículos em cada status (Disponíveis, Em Viagem, Manutenção).**
        - **Média de capacidade de carga da frota cadastrada.**

Use os seguintes dados de entrada:

C

```c
#include <stdio.h>

#define NUM_CIDADES 5

// Nomes das cidades apenas para referência nos menus e relatórios
const char *NOME_CIDADES[NUM_CIDADES] = {
    "0: Central",
    "1: Norte",
    "2: Sul",
    "3: Leste",
    "4: Oeste"
};

// Matriz de distâncias em km
// 0  = Mesma cidade
// -1 = Sem rota direta
int distancias[NUM_CIDADES][NUM_CIDADES] = {
    /*            0:Central  1:Norte  2:Sul  3:Leste  4:Oeste */
    /* 0:Central */ {   0,     45,    60,     30,     80 },
    /* 1:Norte   */ {  45,      0,    -1,     25,    110 },
    /* 2:Sul     */ {  60,     -1,     0,     70,     40 },
    /* 3:Leste   */ {  30,     25,    70,      0,     -1 },
    /* 4:Oeste   */ {  80,    110,    40,     -1,      0 }
};
```